

Software Architecture & Design

Final Report

By Group 1

Contents

1. Group Members	
2. Sprint 1 Architecture	
i. Component Specification Diagram	
3. Sprint 2 Architecture	
i. Business Concept Model	
ii. Business Type Model	
iii. Use Case Diagram	
iv. Business Interfaces	
v. System Interfaces	
vi. Initial System Architecture	
4. Sprint 3 Architecture	
i. Compound Components	
ii. Applied Domain Independent Styles	
iii. Applied SOA Principles	
5. Test Cases	
6. Meeting Records	
7. GitHub Links	

Group Members

Amel Soumia Bouras, A00018158 *Project Owner*

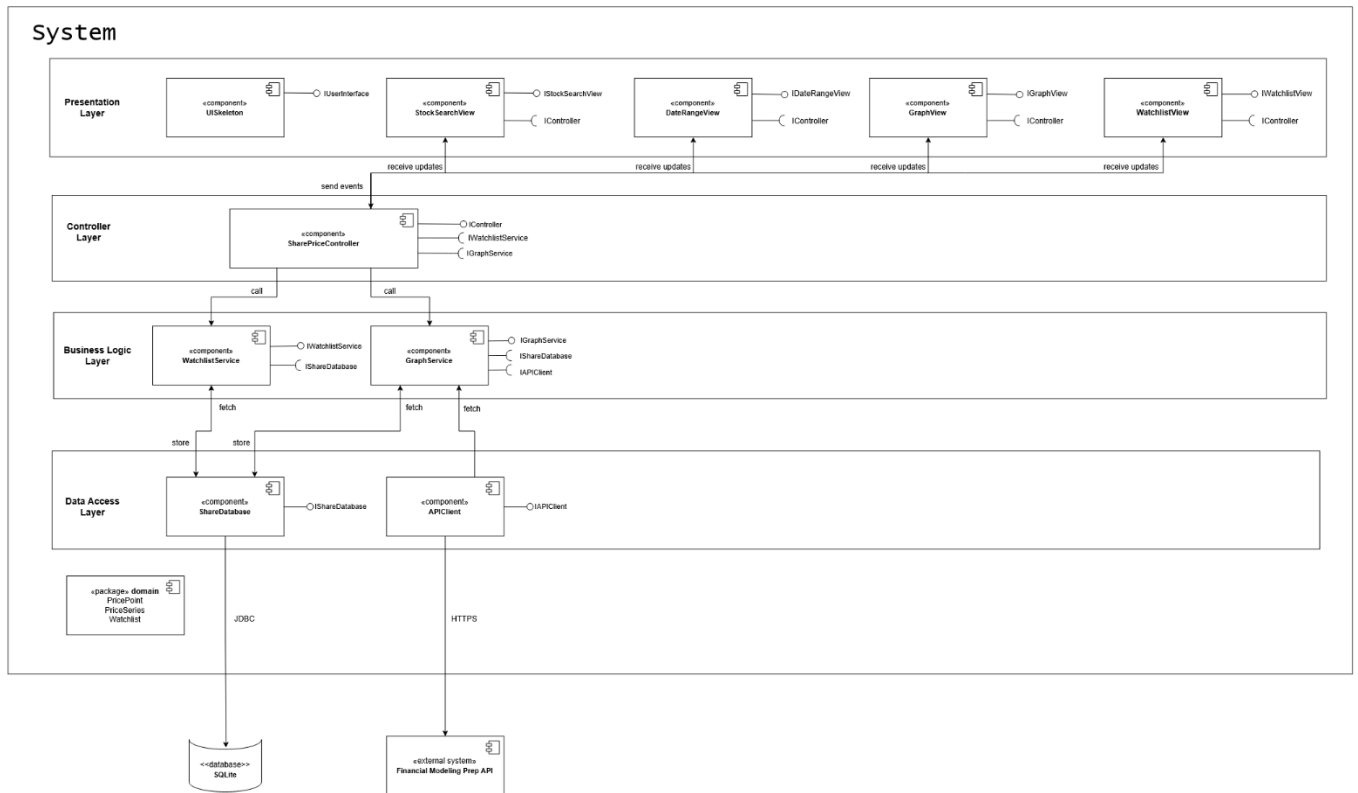
Lyba Ahmad, A00012095 *Scrum Master*

Jochebed Nsiah Saah, NSI22527318 *Team Member*

Rania Malik, A00011818 *Team Member*

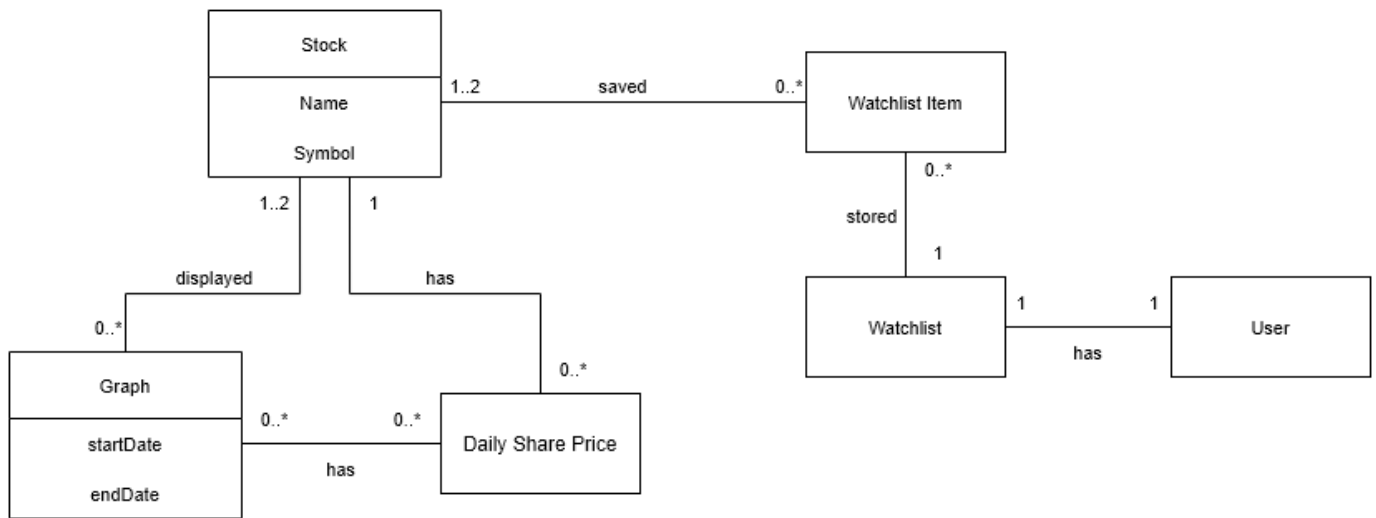
Sprint 1 Architecture

Component Specification Diagram

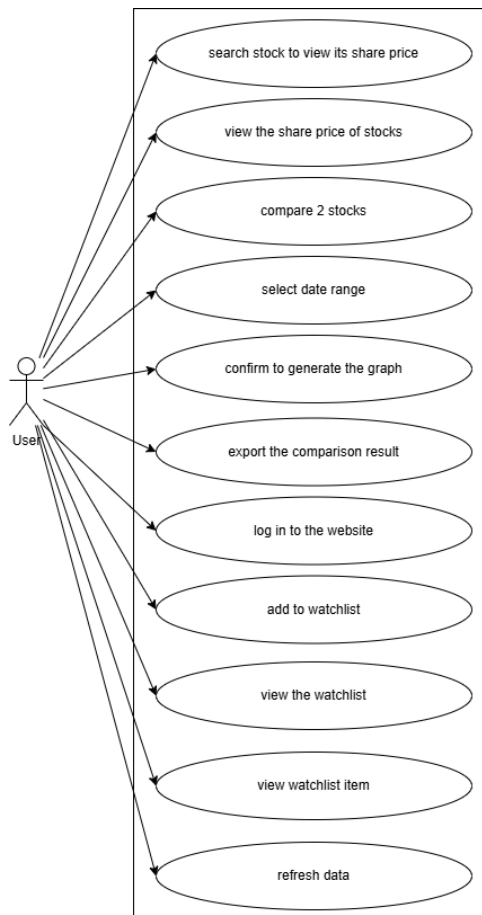


Sprint 2 Architecture

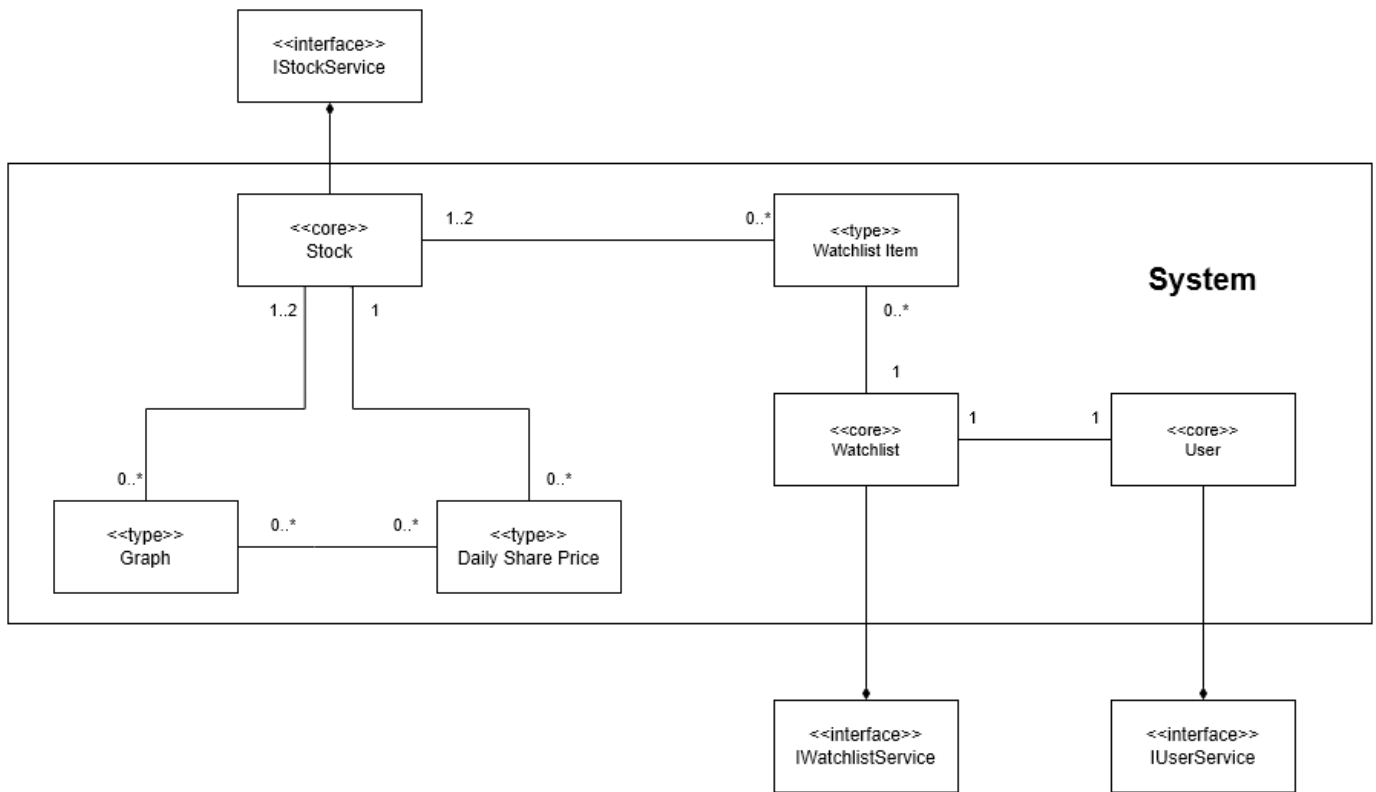
Business Concept Model



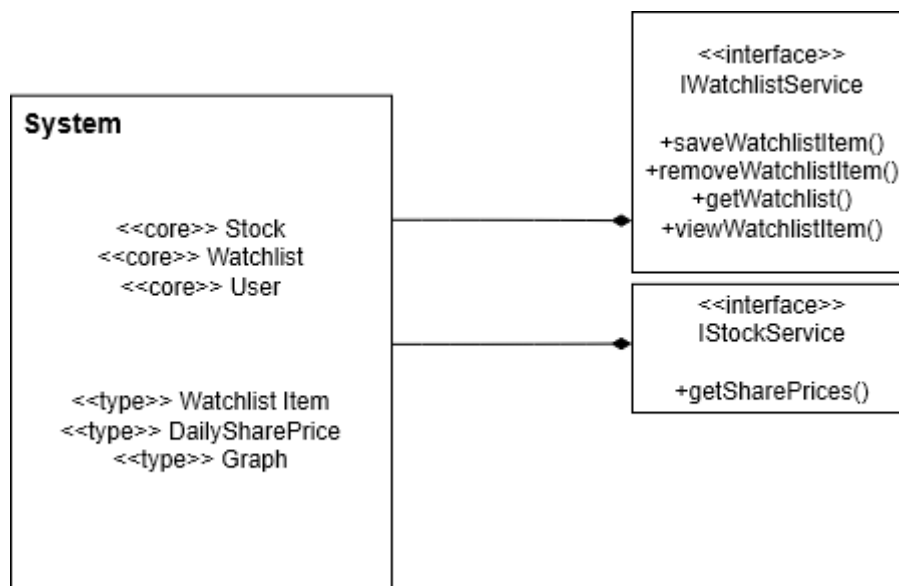
Use Case Diagram



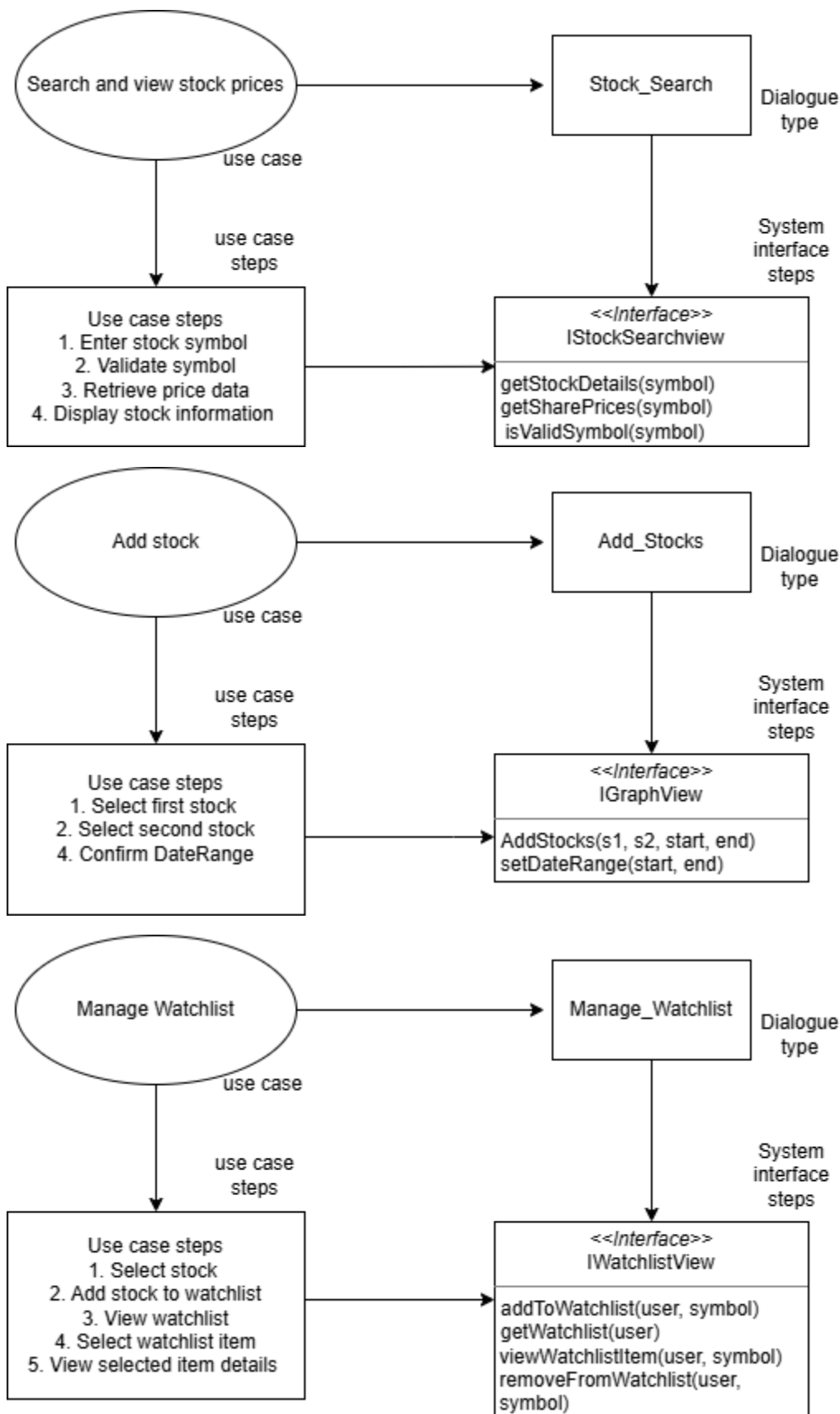
Business Type Model



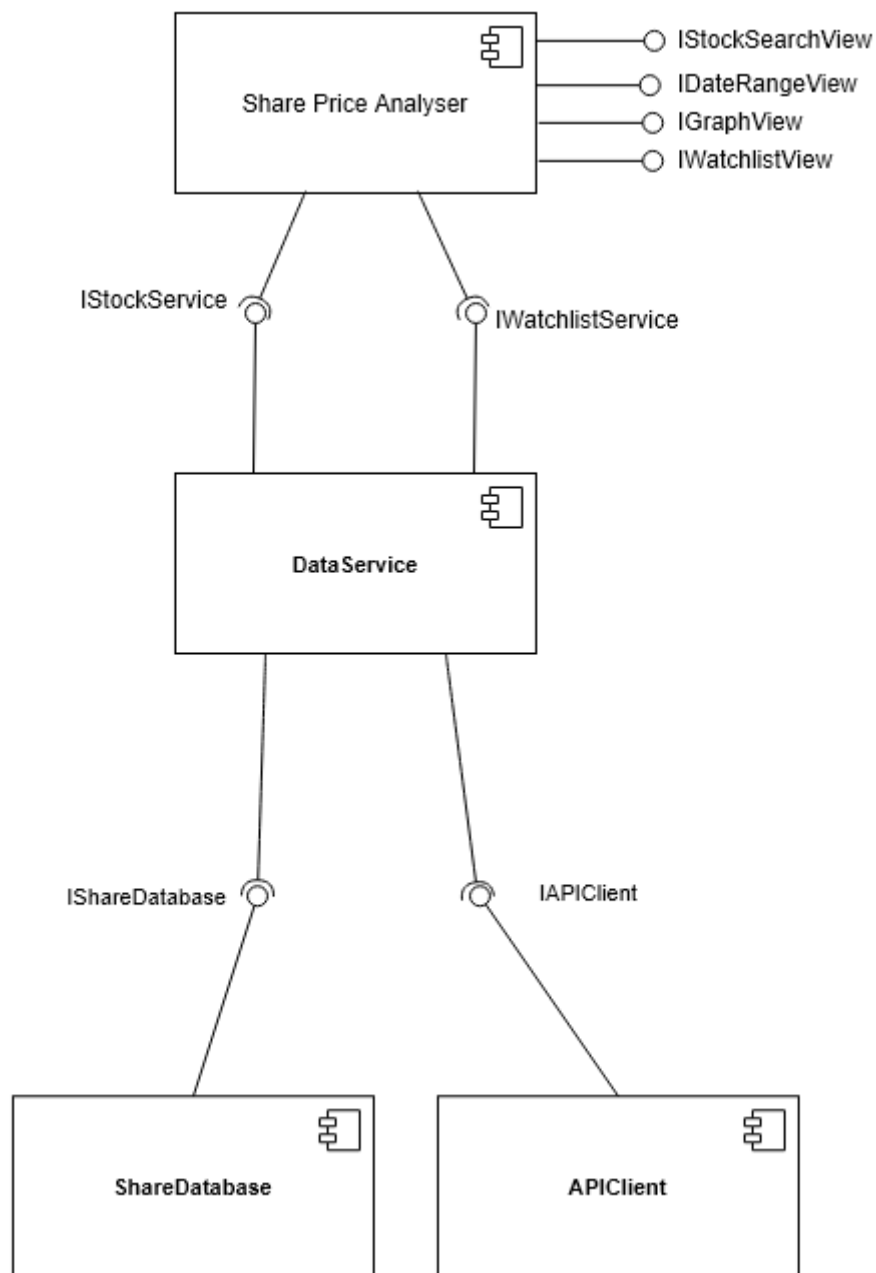
Business Interfaces

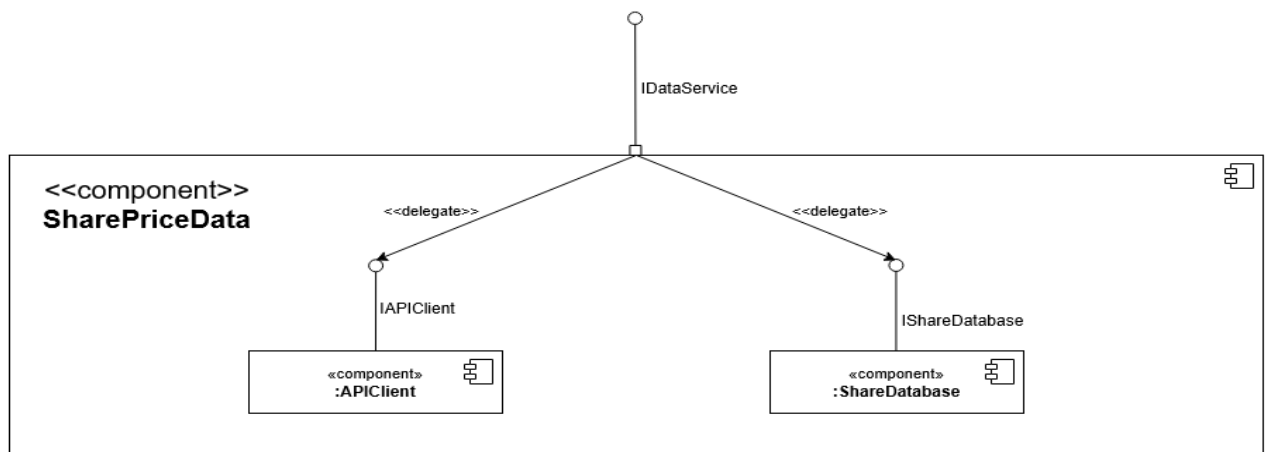
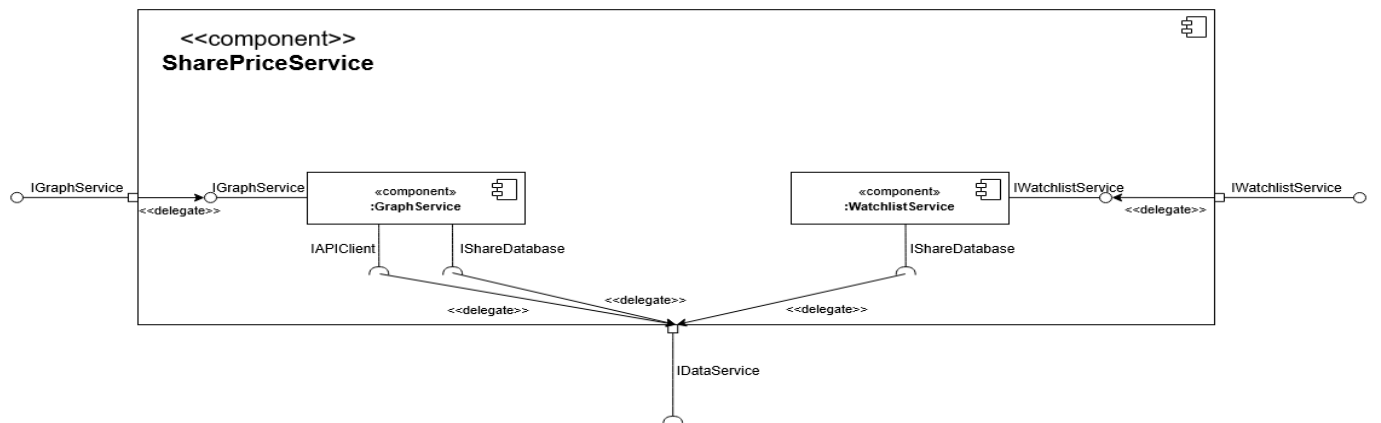
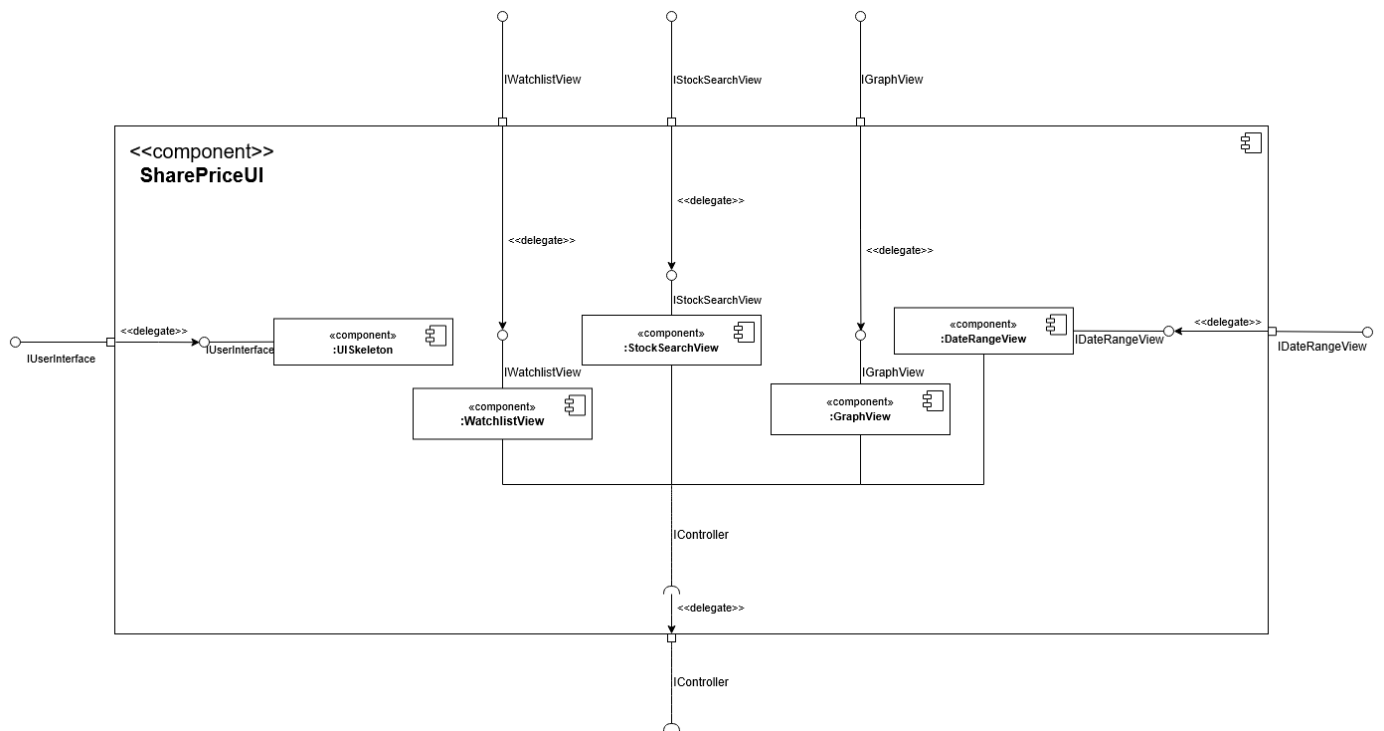


System Interfaces



Initial System Architecture





Compound Component	Justification
SharePriceUI	All components encapsulated share a cohesive set of functions; they handle UI and user input, residing in the presentation layer. Most also require the same interface, IController, so having one compound component that interacts with SharePriceController, rather than multiple, is cleaner and improves modularity.
SharePriceService	Both components encapsulated handle business logic and share a required interface, IShareDatabase. It is therefore cleaner and more modular to combine them into one compound component that interacts with the data access layer (SharePriceData).
SharePriceData	Both components encapsulated handle and provide data from external sources (database, API). It is therefore cleaner and more modular to combine them into one compound component that provides one clear interface to the business logic layer (SharePriceService), maximising encapsulation.

Applied Domain Independent Styles

Our system was originally designed using a **layered** architecture, specifically **N-tier architecture** as it has 4 clear layers with separate responsibilities. **MVC** is also partially applied within these layers as the presentation layer acts as the 'view', SharePriceController as the 'controller', and the rest of the layers contribute to the 'model' responsibility. These were all applied to improve modularity which in turn creates a more reusable and maintainable system. This also helps separate the code to facilitate independent creation of the user interface and behind the scenes logic.

APIClient acts as an **Adapter** as it translates data taken from the external API to a readable format for the components in the business logic layer, completely encapsulating these functions within and only exposing the data through its provided interface. This further contributes to the clean architectural design as changing the API would have little affect on the system, making it replaceable.

The **Pipes and Filters** in GraphService contribute to the modularity of our system by encapsulating separate responsibilities into pipes and passing data through them using a filter to ensure data passes smoothly between the data access layer and the controller.

Applied SOA Principles

Although SOA principles weren't fully implemented, the use of our compound components and their exposed interfaces like 'IDataService' reflect the intended purpose; components of each layer expose the necessary data/functions while implementation details are hidden/encapsulated. Independent deployment wasn't necessary for our small project as it would overcomplicate the system, however, the benefits of SOA such as loose coupling, reusability and clear service boundaries, are reflected in our architecture.

Test Cases

Test Case	Action	Expected Result
Search stock	Type stock name or symbol in search bar, attempt empty search	Symbol suggestions appear (if provided by API), empty search returns nothing
Select stock	Select a symbol from the search suggestions	Correct stock symbol is returned, later displayed on the graph
Invalid date range	Exceeding the two year date range limit	Error message should show, nothing else returned or displayed
Invalid date input	Enter invalid date	Error message should show, nothing else returned or displayed
Select more than 2 stocks	Try selecting 3 stocks	Shouldn't be possible to search or add more than 2, or error message is shown and selection is ignored
Sumit all valid, expected inputs	Select a stock (or 2), and valid date ranges	Stocks displayed with correct information as a line graph
Add to watchlist	Click add to watchlist button	Watchlist item appears in watchlist page
View watchlist	Click on watchlist	Watchlist appears with all added items
Select from watchlist	Click a watchlist item	Correct graph/ item is loaded on the home page

Meeting Records

The records below highlight key group meetings made via WhatsApp call; other meetings include undocumented in-person and text discussions which occurred regularly during each sprint to ensure everyone was on track and on the same page.

Meeting 1

Date and Time	15-02-2026 12:00 – 12:55
Meeting Goal	• Agree on Code of Conduct • Assign roles in the team • Discuss and distribute tasks amongst members
Attendees	Amel, Lyba, Rania, Jochebed
Roundtable Updates	First team meeting so no new updates.
Discussion points	• Code of conduct • Component specification diagram • Researching architecture aspects • Deciding SCRUM Master and Product Owner • Product backlog and requirements
Actions	Code of Conduct – <i>Rania</i> Component diagram – <i>Lyba</i> Product backlog and requirements – <i>Amel</i> Research architectural designs – <i>Jochebed</i>

Meeting 2

Date and Time	17-02-2026 21:15 – 22:45
Meeting Goal	• Verify and finalise Component Specification Diagram • Distribute components amongst members to code
Attendees	Amel, Lyba, Rania, Jochebed
Roundtable Updates	Amel completed the requirements, Jochebed submitted initial research which Lyba used to draft a Component Specification diagram. Rania completed the Code of Conduct.
Discussion points	• Components and structure of the Component Specification Diagram • Initial talks about how to approach code implementation
Actions	Code Business Logic Layer component – <i>Rania</i> Code Presentation Layer components – <i>Lyba</i> Code Data Access Layer components – <i>Amel</i> Code Controller Layer component – <i>Jochebed</i>

Meeting 3

Date and Time	24/02/26 20:30- 21:00
----------------------	--------------------------

Meeting Goal	• Discuss Sprint 2 • Divide and assign tasks
Attendees	Amel, Lyba, Rania, Jochebed
Roundtable Updates	No new updates, reflecting on sprint 1.
Discussion points	• Sprint 2 task explanation • Consideration of changes to the original component diagram using feedback • Assigning components to each member based on new architecture • Dividing diagrams across members
Actions	<i>Amel</i>- business concept model & initial system architecture <i>Lyba</i>- use case & business interfaces <i>Rania</i>- system & business interfaces (1/2) <i>Jochebed</i>- business type model Everyone works on same layer as last sprint, making sure to align with simple architecture and updated business concept.

Meeting 4

Date and Time	13/03/26 20:00- 21:00
Meeting Goal	• Update on progress • Discuss improvements and ensure design consistency • Discuss integration into code
Attendees	Amel, Lyba, Jochebed
Roundtable Updates	Most diagrams complete, except final architecture.

Discussion points	- Progress each member made in sprint 2 - Improvements to diagrams for consistency and accuracy - Changes made to design that impact code, what to update and integrate
Actions	Complete outstanding diagrams and use feedback to update diagrams before code integration. Additional components assigned evenly among members.

Meeting 5

Date and Time	15/04/26 11:00 – 11:15
Meeting Goal	- Plan Sprint 3 - Split tasks between members
Attendees	Amel, Lyba, Rania, Jochebed
Roundtable Updates	No updates, reflecting on Sprint 2.
Discussion points	- Updated architecture choices - Tasks to assign to each member
Actions	Members to work on the same components as Sprint 2, fully implementing them and launching the website. <i>Amel</i> - Report, including Sprint 3 diagrams <i>Lyba</i> - Presentation slides

GitHub Links

Repository:

<https://github.com/Joch1n/share-price-analyser>

Kanban Board:

<https://github.com/users/Joch1n/projects/1>